# Centralized Log Monitoring for a Cowrie Honeypot

Using Promtail, Loki, and Grafana

Nandhu

August 4, 2026

## Contents

**1. Project Overview** **2**

**2. Architecture** **2**

**3. The Data Flow, Explained** **2**

**4. Honeypot Setup (Cowrie)** **4**

**5. Log Server Setup (Loki + Grafana)** **6**

**6. Promtail Setup (on the Honeypot)** **8**

**7. Verification** **9**

**8. Dashboard** **9**

**9. Restart Checklist (After Shutdown)** **10**

**10. Key Takeaways** **10**

# 1. Project Overview

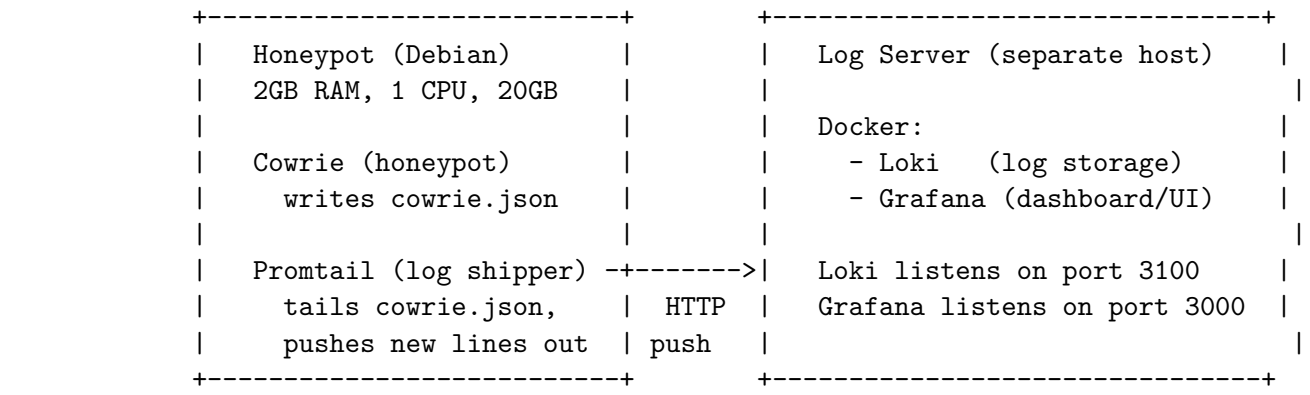
This project deploys a Cowrie SSH/Telnet honeypot on a lightweight Debian system, and builds a centralized, self-hosted log monitoring pipeline for it using **Promtail**, **Loki**, and **Grafana** — a resource-light alternative to heavier SIEM stacks (Graylog, ELK) that would not run acceptably on constrained hardware.

**Goals:**

- Deploy a low-interaction honeypot to capture attacker behavior
- Ship honeypot logs off the honeypot box to a separate, centralized log server (separation of concerns / basic security hygiene)
- Store and index logs efficiently using Loki
- Visualize and query logs in real time using Grafana

# 2. Architecture

Two machines are used, deliberately separating the honeypot (log *source*) from the log server (log *storage and analysis*):




**Why two machines instead of one:** the honeypot should remain lightweight and disposable. If it were ever genuinely compromised, the monitoring stack should not live on the same box. This mirrors real SOC design, where sensors/endpoints are separate from the central monitoring system.

**Why Loki + Grafana instead of Graylog or ELK:** Graylog and Elasticsearch-based stacks typically require a minimum of 4GB+ RAM just to run acceptably, since they bundle a heavyweight search index (Elasticsearch/OpenSearch) alongside the log server itself. Loki was purpose-built to be lighter — it indexes only log *metadata* (labels) rather than full-text indexing every log line, which keeps its resource footprint small enough to run comfortably on modest hardware.

# 3. The Data Flow, Explained

Understanding *why* each piece exists matters more than memorizing commands. The pipeline has five stages:

1. **Attack/Activity** — someone (or a test) connects to the honeypot's SSH/Telnet ports.
2. **Cowrie logs it** — Cowrie fakes the login/shell and records every connection, login attempt, and command as a line of JSON in `cowrie.json`. At this point the data is just sitting on

disk.

3. **Promtail ships it** — Promtail is a small Go binary whose only job is to tail `cowrie.json` and push new lines over HTTP to Loki, as soon as they appear.
4. **Loki stores it** — Loki receives pushed log lines, indexes them by label (e.g. `job="cowrie-honeypot"`), and stores them so they can be queried later.
5. **Grafana displays it** — Grafana holds no log data itself; it is purely a query/visualization frontend. When a search is run in Grafana's Explore view, Grafana forwards that query to Loki, which returns matching results.

## 4. Honeypot Setup (Cowrie)

Performed on the Debian honeypot VM (2GB RAM, 1 CPU, 20GB disk).

```
# Install dependencies
sudo apt update
sudo apt install -y git python3-venv python3-dev libssl-dev \
    libffi-dev build-essential rsyslog

# Create a dedicated non-root user for Cowrie
sudo adduser --disabled-password cowrie
sudo su - cowrie

# Clone and set up Cowrie
git clone http://github.com/cowrie/cowrie
cd cowrie
python3 -m venv cowrie-env
source cowrie-env/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

**Configuration** — the override config lives at the top-level `etc/cowrie.cfg` inside the cloned repo (not inside `src/cowrie/data/etc/`, which only holds the bundled template):

```
[honeypot]
hostname = prod-web-01
state_path = var/lib/cowrie
log_path = var/log/cowrie

[ssh]
enabled = true
listen_endpoints = tcp:2222:interface=0.0.0.0
version = SSH-2.0-OpenSSH_8.4p1 Debian-5

[telnet]
enabled = true
listen_endpoints = tcp:2223:interface=0.0.0.0

[shell]
filesystem = share/cowrie/fs.pickle

[output_jsonlog]
enabled = true
logfile = var/log/cowrie/cowrie.json
epoch_timestamp = false

[output_localsyslog]
enabled = false
```

**Redirect real ports 22/23 to Cowrie's 2222/2223**, so it presents like a normal exposed

SSH/Telnet service to a scanner:

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 22 \
    -j REDIRECT --to-port 2222
sudo iptables -t nat -A PREROUTING -p tcp --dport 23 \
    -j REDIRECT --to-port 2223
```

**Start Cowrie:**

```
bin/cowrie start
bin/cowrie status
```

Logs are written to: `var/log/cowrie/cowrie.json`

## 5. Log Server Setup (Loki + Grafana)

Performed on a separate host (an 8GB laptop in this deployment), using Docker to run both services together.

```
sudo apt update
sudo apt install -y docker.io docker-compose
sudo systemctl enable docker
sudo systemctl start docker
```

**loki-config.yaml** — minimal single-node configuration using local filesystem storage (no external database required):

```
auth_enabled: false

server:
  http_listen_port: 3100

common:
  path_prefix: /loki
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules
  replication_factor: 1
  ring:
    instance_addr: 127.0.0.1
    kvstore:
      store: inmemory

schema_config:
  configs:
    - from: 2020-10-24
      store: boltdb-shipper
      object_store: filesystem
      schema: v11
      index:
        prefix: index_
        period: 24h

ruler:
  alertmanager_url: http://localhost:9093
```

**docker-compose.yml** — brings up Loki and Grafana together:

```
version: "3.8"

services:
  loki:
    image: grafana/loki:2.9.0
```

```
    container_name: loki
    ports:
      - "3100:3100"
    volumes:
      - ./loki-config.yaml:/etc/loki/local-config.yaml
      - loki-data:/loki
    command: -config.file=/etc/loki/local-config.yaml
    restart: unless-stopped

  grafana:
    image: grafana/grafana:10.4.0
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=changeme123
    volumes:
      - grafana-data:/var/lib/grafana
    restart: unless-stopped
    depends_on:
      - loki

volumes:
  loki-data:
  grafana-data:
```

**Bring the stack up:**

```
cd ~/loki-grafana
sudo docker-compose up -d
sudo docker ps
curl http://localhost:3100/ready    # should return "ready"
```

**Add Loki as a Grafana data source:**

1. Open `http://<log_server_ip>:3000`, log in with `admin` / `changeme123` (change on first login)
2. Left sidebar -> Connections -> Data sources -> Add data source
3. Select **Loki**
4. URL: `http://loki:3100` (resolves via the shared Docker network)
5. Click **Save & test** — should confirm connection

## 6. Promtail Setup (on the Honeypot)

Promtail is a lightweight (~50MB RAM) log shipper — it runs alongside Cowrie without meaningfully competing for resources.

```
sudo apt install -y unzip

curl -O -L "https://github.com/grafana/loki/releases/download/v2.9.0/promtail-linux-amd64.zip"
unzip promtail-linux-amd64.zip
sudo mv promtail-linux-amd64 /usr/local/bin/promtail
chmod +x /usr/local/bin/promtail

sudo mkdir -p /etc/promtail
```

**promtail-config.yaml** (/etc/promtail/promtail-config.yaml):

```
server:
  http_listen_port: 9080
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://<log_server_ip>:3100/loki/api/v1/push

scrape_configs:
  - job_name: cowrie
    static_configs:
      - targets:
          - localhost
        labels:
          job: cowrie-honeypot
          host: honeypot-01
          __path__: /home/server/cowrie/var/log/cowrie/cowrie.json
```

**Run Promtail as a systemd service** (so it survives reboots):

```
sudo tee /etc/systemd/system/promtail.service << 'EOF'
[Unit]
Description=Promtail log shipper
After=network.target

[Service]
ExecStart=/usr/local/bin/promtail -config.file=/etc/promtail/promtail-config.yaml
Restart=always

[Install]
WantedBy=multi-user.target
EOF
```

```
sudo systemctl daemon-reload
sudo systemctl enable promtail
sudo systemctl start promtail
sudo systemctl status promtail
```

## 7. Verification

From the honeypot box, confirm Loki is reachable:

```
curl http://<log_server_ip>:3100/ready
```

In Grafana, go to **Explore**, select the Loki data source, and run:

```
{job="cowrie-honeypot"}
```

Live Cowrie log lines should appear. A filtered query for failed login attempts:

```
{job="cowrie-honeypot"} |= "login.failed"
```

## 8. Dashboard

A basic monitoring dashboard was built with two panels:

- **Failed logins over time**: `count_over_time({job="cowrie-honeypot"} |= "login.failed" [5m])`
- **Live connection log**: `{job="cowrie-honeypot"} |= "session.connect"`

## 9. Restart Checklist (After Shutdown)

Bring the Log Server up first, since Promtail needs somewhere to send data to.

**On the Log Server:**

```
cd ~/loki-grafana
sudo docker-compose up -d
sudo docker ps
curl http://localhost:3100/ready
```

**On the Honeypot:**

```
cd /home/server/cowrie
source cowrie-env/bin/activate
bin/cowrie start
bin/cowrie status

sudo systemctl status promtail   # should already be running
```

**Confirm end-to-end connectivity:**

```
curl http://<log_server_ip>:3100/ready
```

Then check Grafana Explore for fresh log activity.

## 10. Key Takeaways

- Centralized logging separates **log generation** (honeypot) from **log storage/analysis** (log server) — both for resource reasons and basic security hygiene.
- Loki's label-based indexing (rather than full-text indexing like Elasticsearch) is what keeps this stack lightweight enough for constrained hardware.
- The pipeline is modular: any of the three components (Promtail, Loki, Grafana) could be swapped for an alternative without redesigning the others, since they communicate over simple, well-defined HTTP APIs.